JAVA 2

Module EXC

Exceptions

Au programme

1. Le mécanisme des exceptions
2. Lever et déclarer une exception
3. Attraper les exceptions
4. Les exceptions non contrôlées

1. Un objet qui interrompt



Démonstration



Démonstration

Faire planter, lire la trace, corriger

Déroulé : Demo_EXC

Vous levez déjà des exceptions

Chaque squelette du cours contient déjà cette ligne

La fabrique de G.15 aussi

Ce module vous permettra d'en savoir plus sur les exceptions et leurs règles

```
public int etageCourant() {
    throw new UnsupportedOperationException(
            "etageCourant n'est pas encore implementee");
}
```

Une exception est un objet

Un objet, créé par `new`, d'une classe qui hérite de `Throwable`

Une `Error` signale que la machine elle-même est en difficulté

Une `Exception` signale ce que le programme peut traiter

```
Throwable
 |-- Error                    > ne s'attrape pas
 |-- Exception                > controlee
      |-- RuntimeException    > non controlee
```

Les exceptions remontent la pile

Le `throw` interrompt la méthode à l'instant même où il est exécuté

L'exception remonte d'appelant en appelant, jusqu'à un `catch`

Sans `catch`, le programme s'arrête et affiche la trace

```
void monter(int etage) {
    verifier(etage);        // l'exception traverse monter
    deplacer(etage);        // jamais execute
}

void verifier(int etage) {
    if (etage > dernierEtage) {
        throw new IllegalArgumentException("Etage " + etage
                + " au-dela du dernier, " + dernierEtage);
    }
}
```

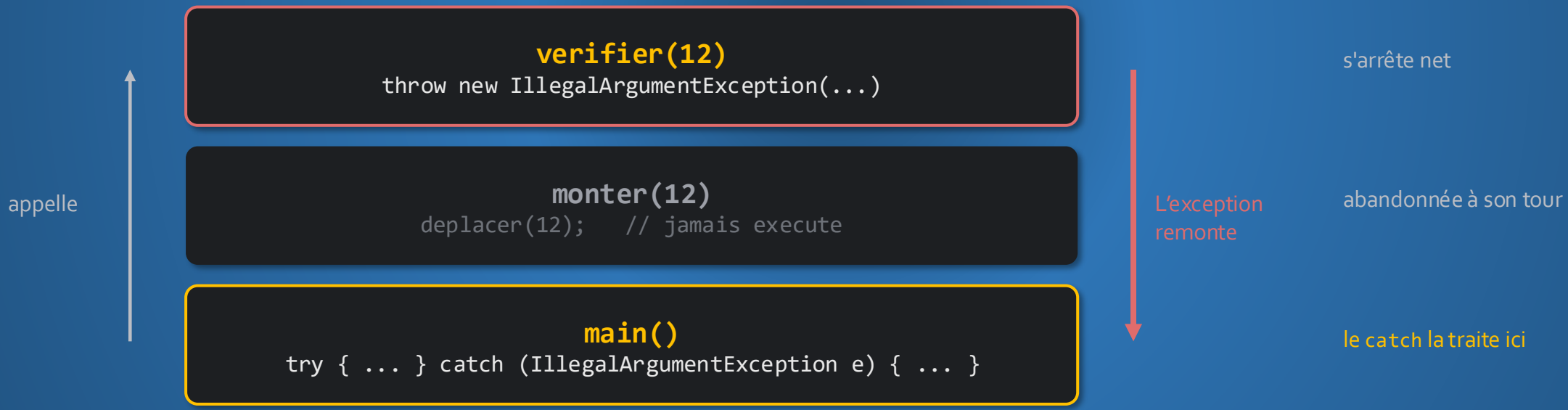
Du `throw` au `catch`

Chaque appel empile une méthode : `main` appelle `monter`, qui appelle `verifier`

L'exception levée dans `verifier` remonte par `monter`, abandonnée, jusqu'au `catch` de `main`

```java
void monter(int etage) {
  verifier(etage);
  deplacer(etage);
}

void verifier(int etage) {
  if (etage > dernierEtage) {
    throw new IllegalArgumentException(...);
  }
}
```



2. Lever, et déclarer



Écrire sa classe d'exception

Une exception est juste une classe qui hérite d'`Exception`

Elle présente un constructeur qui reçoit le message

…et qui le transmet à la classe mère par `super`

```
public class EtageInaccessibleException extends Exception {

    public EtageInaccessibleException(String message) {
        super(message);
    }
}
```

Lever avec `throw`, déclarer avec `throws`

Le mot `throw` lève l'exception, dans le corps de la méthode

Le mot `throws` l'annonce, dans la signature de la méthode

Un constructeur annonce une exception exactement de la même façon

```
public void appeler(int etage) throws EtageInaccessibleException {
    if (etage < 0 || etage > dernierEtage) {
        throw new EtageInaccessibleException("Etage " + etage
                + " hors de l'intervalle 0 a " + dernierEtage);
    }
    etageCourant = etage;
}
```

Attraper ou déclarer

Pour une exception contrôlée, le compilateur exige un choix

- Attraper : la traiter ici, avec `try` et `catch`
- Déclarer : la laisser passer à l'appelant, qui aura le même choix

```
void depuisLeBouton(int etage) {
    try {
        ascenseur.appeler(etage);
    } catch (EtageInaccessibleException e) {
        System.out.println("Refus : " + e.getMessage());
    }
}

void depuisLeBadge(int etage) throws EtageInaccessibleException {
    ascenseur.appeler(etage);
}
```

Exercices



EXC.1 Lever sa propre exception

fascicule EXC

EXC.2 Un compte, sans exception

fascicule EXC

Messages : CC-ERR.4

3. Attraper là où l'on peut agir



Ce que le `try` saute

Au premier `throw`, le reste du `try` est abandonné

Le `catch` s'exécute, puis tout ce qui le suit

Sans exception, le `catch` est simplement ignoré

```
public void appeler(int etage)
                throws EtageInaccessibleException {
    if (etage < 0 || etage > dernierEtage) {
        throw new EtageInaccessibleException("Etage "
            + etage + " hors de l'intervalle 0 a "
            + dernierEtage);
    }
    etageCourant = etage;
}
```

```
try {
    ascenseur.appeler(2);
    ascenseur.appeler(9);              // leve l'exception
    System.out.println("Arrive");      // jamais affiche
} catch (EtageInaccessibleException e) {
    System.out.println("Refus : " + e.getMessage());
}
System.out.println("Fin");             // toujours affiche
```

Plusieurs `catch`, ou un seul

Un `try` peut être suivi de plusieurs `catch`

Un `catch` attrape aussi les sous-classes de son type

Une barre verticale réunit deux types dans le même `catch`

```
try {
    ascenseur.appeler(etage);
    porte.ouvrir();
} catch (EtageInaccessibleException e) {
    afficher("Etage refuse : " + e.getMessage());
} catch (PorteBloqueeException | CoupureException e) {
    signalerPanne(e.getMessage());
}
```

Un `catch` fait quelque chose

Un `catch` vide cache l'erreur sans la résoudre

Traiter, informer, ou ne pas attraper du tout

Règle `CC-ERR.6`

```
try {
    ascenseur.appeler(etage);
} catch (EtageInaccessibleException e) {
    System.out.println("Refus : " + e.getMessage());
    ecran.proposerEtagesValides();
}
```

Ce que `finally` garantit

Le bloc `finally` s'exécute toujours, exception ou non

Même quand l'exception n'est pas attrapée

Il sert à remettre les choses en place

```
void deplacer(int etage) throws EtageInaccessibleException {
    porte.verrouiller();
    try {
        ascenseur.appeler(etage);
    } finally {
        porte.deverrouiller();          // dans tous les cas
    }
}
```

Exercices



EXC.3 Un numéro de compte vérifié

fascicule EXC

EXC.6 Ce que `finally` garantit

fascicule EXC

Exceptions : `CC-ERR.4`, `CC-ERR.5`

4. Celles que le compilateur ne voit pas



Une exception que personne n'annonce

Une `RuntimeException` n'exige ni `try` ni `throws`

Le code compile donc sans un mot

Et le programme s'arrête à l'exécution

```
String saisie = "douze";
int etage = Integer.parseInt(saisie);   // compile sans rien dire

// Exception in thread "main" java.lang.NumberFormatException:
//     For input string: "douze"
```

`NullPointerException` : la plus fréquente

Une méthode appelée sur `null` : pas d'objet à qui demander

Avec `&&`, l'ordre des tests compte

Mieux : ne pas produire de `null` (`CC-ERR.7`)

```
String titre = null;

titre.equals("Nana")                    // plantage

titre != null && titre.equals("Nana")   // false
"Nana".equals(titre)                    // false
Objects.equals(titre, "Nana")           // false
```

Exercices



EXC.4 Provoquer, puis traiter

fascicule EXC

D'abord sans aucun `try`

Réutiliser plutôt que créer

Exception	Elle signale
`IllegalArgumentException`	un paramètre refusé
`IllegalStateException`	un objet dans le mauvais état
`UnsupportedOperationException`	une opération absente

Lever une exception du JDK

Elle se lève comme les autres, sans `throws` à déclarer

Le message dit ce qui était attendu et ce qui a été reçu

Règle `CC-ERR.4`

```
public Ascenseur(int dernierEtage) {
    if (dernierEtage < 0) {
        throw new IllegalArgumentException(
                "Le dernier etage doit etre positif ou nul, pas " + dernierEtage);
    }
    this.dernierEtage = dernierEtage;
}
```

Exercices



EXC.5 Lire une trace

fascicule EXC

Pile d'appels : TEC-EXE.1

Le fil rouge, relu



G.15 La fabrique
fil rouge GameHub

G.24 La méthode jouer, mise en place
fil rouge GameHub

Relire son code avec le vocabulaire du module